

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Интеллектуальный анализ данных»
Тема: «Предобучение нейронных сетей с использованием автоэнкодерного
подхода»

Выполнила:
Студентка 4 курса
Группы ИИ-25
Мархель А.Д.
Проверила:
Андренко К.В.

Цель работы: научиться осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода.

Общее задание

1. Взять за основу любую сверточную или полносвязную архитектуру с количеством слоев более 3. Осуществить ее обучение (без предобучения) в соответствии с вариантом задания. Получить оценку эффективности модели, используя метрики, специфичные для решаемой задачи (например, MAPE – для регрессионной задачи или F1/Confusion matrix для классификационной);
2. Выполнить обучение с предобучением, используя автоэнкодерный подход, алгоритм которого изложен в лекции. Условие останова (например, по количеству эпох) при обучении отдельных слоев с использованием автоэнкодера выбрать самостоятельно;
3. Сравнить результаты, полученные при обучении с/без предобучения, сделать выводы;
4. Выполнить пункты 1–3 для датасетов из ЛР №2;
5. Оформить отчет по выполненной работе, загрузить исходный код и отчет.

Вариант 6

№ варианта	Выборка	Тип задачи	Целевая переменная
6	Physicochemical Properties of Protein Tertiary Structure (UCI, CASP.csv)	регрессия	RMSD
ЛР №2	Wisconsin Diagnostic Breast Cancer (WDBC, wdbc.data)	классификация	Diagnosis (B / M)

Ход работы

1. Выборки

Physicochemical Properties of Protein Tertiary Structure (вариант 6) – данные CASP о пространственной структуре белков. Выборка содержит 45 730 объектов (файл CASP.csv, загружен с UCI Machine Learning Repository), 9 вещественных физико-химических признаков F1–F9 (площадь поверхности, площадь неполярной части, масса, штраф вторичной структуры и др.) и целевую переменную RMSD – среднеквадратичное отклонение предсказанной структуры от нативной (в ангстремах). Это регрессионная задача. Пропусков нет; RMSD изменяется от 0 до 21 Å (среднее 7,75 Å), в выборке 272 объекта с RMSD = 0. Фрагмент файла:

RMSD	F1	F2	F3	F4	F5	F6	F7	F8	F9
17.284	13558.3	4305.35	0.31754	162.173	1872790.5	215.359	4287.87	102	27.0302
6.021	6191.96	1623.16	0.26213	53.3894	803446.7	87.2024	3328.91	39	38.5468

WDBC (Wisconsin Diagnostic Breast Cancer) – набор из ЛР №2: 569 объектов (357 доброкачественных В и 212 злокачественных М), 30 вещественных признаков (10 параметров ядра клетки в виде среднего, стандартной ошибки и худшего значения). Это задача бинарной классификации; положительным классом считается М. Файл wdbc.data взят из ЛР №2 без изменений.

2. Подготовка данных

Выборка случайно разбивается на обучающую, валидационную и тестовую части в пропорции 60 / 20 / 20 (для WDBC – со стратификацией по классам): Protein – 27 438 / 9 146 / 9 146 объектов, WDBC – 341 / 114 / 114. Параметры стандартизации (среднее и СКО) вычисляются только по обучающей части и применяются ко всем. Для регрессии целевая переменная RMSD также стандартизуется, а все метрики пересчитываются обратно в ангстремы. Обучающая часть используется для предобучения и обучения, валидационная – для выбора лучшей эпохи и построения кривых, тестовая – только для итоговой оценки.

Так как на одном разбиении результат зависит от случайности, каждый эксперимент повторяется для нескольких значений seed (Protein – 5, WDBC – 10); seed определяет и разбиение, и начальные веса. Для каждого seed обе модели – без предобучения и с ним – стартуют из одних и тех же случайных весов, на одном разбиении, с одинаковым оптимизатором, порядком мини-батчей и числом эпох тонкой настройки. Различие между ними только в том, что вторая модель прошла предобучение.

3. Архитектура сети

Выборка	Архитектура (слои)	Слоёв с весами	Активация скрытых слоёв	Выход	Функция потерь
Protein (регрессия)	9-128-64-32-16-1	5 (4 скрытых)	ReLU	1 линейный нейрон	MSE
WDBC (классификация)	30-64-32-16-8-2	5 (4 скрытых)	ReLU	2 логита (softmax в CE)	Cross-Entropy

В обоих случаях используется полносвязная сеть с четырьмя скрытыми слоями и выходным слоем (всего 5 слоёв с настраиваемыми весами, что больше 3 и удовлетворяет определению глубокой сети из лекции). Функции потерь

выбраны по рекомендации лекции: MSE – для регрессии, Cross-Entropy – для классификации. Функция активации ReLU выбрана заранее, до экспериментов, как рекомендованная лекцией для глубоких сетей; логистическая функция и гиперболический тангенс исследованы отдельно (п. «Влияние функции активации»).

4. Предобучение автоэнкодерами

Реализован алгоритм жадного послойного предобучения из лекции (функция `pretrain_layers`, файл `deep_net.py`):

1. для первого скрытого слоя строится автоассоциативная сеть $X \rightarrow Y_1 \rightarrow X$: кодером служит сам слой исходной сети (`nn.Linear` + `ReLU`), декодером – вспомогательный линейный слой; сеть обучается без учителя по MSE реконструкции, после чего веса W_1 фиксируются;
2. обучающая выборка пропускается через обученный слой, и выходы Y_1 подаются на вход второго автоэнкодера $Y_1 \rightarrow Y_2 \rightarrow Y_1$; так получают веса W_2 ;
3. процесс повторяется для всех четырёх скрытых слоёв, реконструирующие слои отбрасываются;
4. выходной слой обучается с учителем (MSE или CE) при замороженных скрытых слоях – 10 эпох для Protein, 20 для WDBC;
5. вся сеть дообучается (fine-tuning) методом обратного распространения ошибки.

Условия останова выбраны самостоятельно: каждый автоэнкодер обучается фиксированное число эпох – 30 для Protein (27 438 объектов) и 100 для WDBC (341 объект), что не превышает рекомендованные в лекции 100 эпох; оптимизатор Adam, $lr = 0,001$, размер мини-батча $\max(16, \min(256, N/8))$. Тонкая настройка: Adam, $lr = 0,001$, 100 эпох (Protein) и 150 эпох (WDBC); в качестве итоговой берётся модель с минимальной ошибкой на валидации. В предобучении участвуют только признаки обучающей выборки, целевая переменная не используется.

Код программы:

```
def make_loss(task):
    return nn.MSELoss() if task == "regression" else nn.CrossEntropyLoss()

@torch.no_grad()
def evaluate(model, X, y, split, task):
    """Возвращает loss и метрики; регрессия оценивается в исходных единицах RMSD."""
    model.eval()
    out = model(X)
    loss = make_loss(task)(out, y).item()
    if task == "regression":
```

```

        sc = split["y_scaler"]
        y_pred = sc.inverse_transform(out.numpy()).ravel()
        y_true = sc.inverse_transform(y.numpy()).ravel()
        metrics = regression_metrics(y_true, y_pred)
    else:
        y_pred = out.argmax(dim=1).numpy()
        y_true = y.numpy()
        metrics = classification_metrics(y_true, y_pred)
    return loss, metrics, y_true, y_pred

def _train(model, params, split, task, epochs, lr, seed, record=None):
    """Эпохи обучения Adam по MSE / CE. record - история (val_loss пишется после каждой эпохи)."""
    X, y = split["X_tr"], split["y_tr"]
    gen = torch.Generator().manual_seed(seed + 2000)
    batch = batch_size_for(len(X))
    loss_fn = make_loss(task)
    opt = torch.optim.Adam(params, lr=lr)
    for _ in range(epochs):
        model.train()
        total = 0.0
        for idx in _batches(len(X), batch, gen):
            loss = loss_fn(model(X[idx]), y[idx])
            opt.zero_grad()
            loss.backward()
            opt.step()
            total += loss.item() * len(idx)
        if record is not None:
            record["train_loss"].append(total / len(X))
            record["val_loss"].append(evaluate(model, split["X_va"], split["y_va"], split,
task) [0])
            if record["val_loss"][-1] < record["best_val"]:
                record["best_val"] = record["val_loss"][-1]
                record["best_epoch"] = len(record["val_loss"]) - 1
                record["best_state"] = copy.deepcopy(model.state_dict())

def fit(model, split, task, cfg, seed, pretrained):
    """Обучение с учителем. При pretrained=True сначала обучается только выходной слой
(скрытые слои заморожены), затем настраивается вся сеть. Для сети без предобучения
выходной слой отдельно не обучается - тонкая настройка начинается со случайных весов.
history['val_loss'][0] - ошибка на валидации перед тонкой настройкой."""
    if pretrained and cfg.head_epochs > 0:
        head = linear_layers(model)[-1]
        for p in model.parameters():
            p.requires_grad = False
        for p in head.parameters():
            p.requires_grad = True
        _train(model, list(head.parameters()), split, task, cfg.head_epochs, cfg.lr, seed)
        for p in model.parameters():
            p.requires_grad = True

    v0 = evaluate(model, split["X_va"], split["y_va"], split, task) [0]
    hist = {"train_loss": [], "val_loss": [v0], "best_val": v0, "best_epoch": 0,
            "best_state": copy.deepcopy(model.state_dict())}
    _train(model, list(model.parameters()), split, task, cfg.epochs, cfg.lr, seed, hist)
    hist["final_state"] = copy.deepcopy(model.state_dict())
    model.load_state_dict(hist.pop("best_state")) # модель с минимальной ошибкой на валидации
    return hist

def run_pair(split, task, sizes, act_name, seed, cfg):
    """Обе модели стартуют из одних и тех же случайных весов, одинаковые разбиение,
оптимизатор, порядок мини-батчей и число эпох тонкой настройки."""
    set_seed(seed)
    base = build_mlp(sizes, act_name)

    plain = copy.deepcopy(base)
    h_plain = fit(plain, split, task, cfg, seed, pretrained=False)

```

```

pre = copy.deepcopy(base)
curves = pretrain_layers(pre, split["X_tr"], act_name, cfg, seed) # только train, без
меток
h_pre = fit(pre, split, task, cfg, seed, pretrained=True)

return {"plain": _finish(plain, h_plain, split, task, cfg),
        "pre": _finish(pre, h_pre, split, task, cfg, curves)}

```

```

def pretrain_layers(model, X, act_name, cfg, seed):
    """Жадное послойное предобучение скрытых слоёв автоэнкодерами. Меняет веса model.
    Возвращает кривые MSE реконструкции для каждого автоэнкодера."""
    torch.manual_seed(seed + 1000)
    gen = torch.Generator().manual_seed(seed + 1000)
    batch = batch_size_for(len(X))
    mse = nn.MSELoss()
    H = X
    curves = []
    for enc_lin in linear_layers(model)[-1:]:
        act = ACTIVATIONS[act_name]()
        dec = nn.Linear(enc_lin.out_features, enc_lin.in_features) # реконструирующий слой
        opt = torch.optim.Adam(list(enc_lin.parameters()) + list(dec.parameters()),
                                lr=cfg.ae_lr)
        curve = []
        for _ in range(cfg.ae_epochs):
            total = 0.0
            for idx in _batches(len(H), batch, gen):
                h = H[idx]
                loss = mse(dec(act(enc_lin(h))), h)
                opt.zero_grad()
                loss.backward()
                opt.step()
                total += loss.item() * len(idx)
            curve.append(total / len(H))
        curves.append(curve)
        with torch.no_grad():
            H = act(enc_lin(H)) # выход слоя -> вход следующего автоэнкодера
    return curves

```

5. Метрики и показатели динамики обучения

- **Регрессия (Protein):** RMSE, MAE и R^2 рассчитываются в исходных единицах RMSD на тестовой выборке. MAPE по определению не существует при $RMSD = 0$ и неустойчив при значениях, близких к нулю (в выборке 272 нуля и ещё 1 385 значений меньше 1 Å), поэтому MAPE считается по объектам с $RMSD \geq 1$ Å; дополнительно приводится WARE $= \Sigma|e| / \Sigma|y| \cdot 100$ %, учитывающий все объекты.
- **Классификация (WDBC):** Efficiency, % (доля верных ответов, argmax по выходным нейронам – формула лекции), F1, Precision, Recall для класса М и матрица ошибок (Confusion matrix).
- **Динамика обучения:** (а) ошибка на валидации до начала тонкой настройки (эпоха 0); (б) «колебательность» – средний модуль изменения val loss между соседними эпохами (после 10-й эпохи) в процентах от средней val loss; (в) «шум» – СКО отклонений val loss от скользящего среднего по 5 эпохам, также в процентах от средней val loss (исключает влияние общего тренда, например переобучения); (г) число эпох до

достижения заданного порога val loss. Показатели (б) и (в) отвечают на вопрос о сглаживании процесса обучения.

- **Статистика:** результаты приведены как среднее $\pm$ СКО по запускам; значимость различий проверена парным t-тестом (один seed = одно и то же разбиение и те же начальные веса).

Результат

1. Protein Tertiary Structure (вариант б, регрессия)

Качество на тестовой выборке, среднее $\pm$ СКО по 5 запускам (MAPE – по объектам с $\text{RMSD} \geq 1 \text{ \AA}$):

Режим	RMSE, $\text{\AA}$	MAE, $\text{\AA}$	R^2	MAPE, %	WAPE, %
Без предобучения	3.871 ± 0.063	2.677 ± 0.062	0.599 ± 0.011	53.897 ± 3.729	34.564 ± 0.833
С предобучением	3.908 ± 0.065	2.788 ± 0.047	0.591 ± 0.013	56.434 ± 1.643	35.997 ± 0.545

Парный t-тест: RMSE $p = 0,030$; MAE $p = 0,002$; R^2 $p = 0,031$; MAPE $p = 0,104$; WAPE $p = 0,002$. Предобучение не улучшило качество: сеть без предобучения оказалась чуть точнее (RMSE 3,871 против 3,908 $\text{\AA}$, т.е. на $\approx 1\%$). Разница статистически различима, но практически мала по сравнению с погрешностью самой модели (MAPE $\approx 54\text{--}56\%$, $R^2 \approx 0,6$).

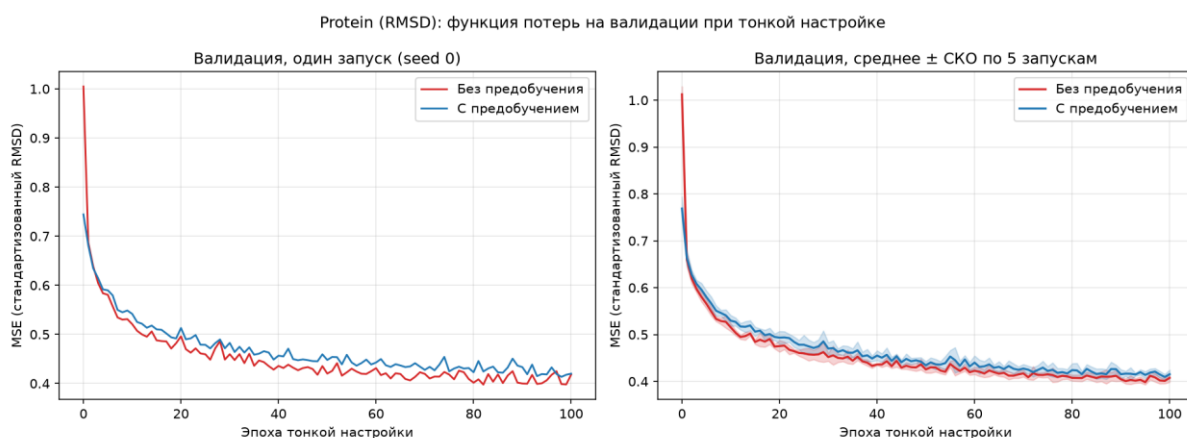


Рисунок 1 – Ошибка на валидации при тонкой настройке (Protein)

Динамика обучения (после 10-й эпохи; показатели в % от средней val loss):

Режим	Val loss, эпоха 0	Лучшая эпоха	Колебательность, %	Шум val-кривой, %	Эпох с ростом val, %
Без предобучения	1.013 ± 0.019	93.200 ± 6.834	2.419 ± 0.186	2.072 ± 0.258	45.333 ± 2.876
С предобучением	0.769 ± 0.028	97.200 ± 2.950	2.266 ± 0.304	1.952 ± 0.532	43.778 ± 2.166

Предобучение заметно снижает стартовую ошибку (0,769 против 1,013 в единицах дисперсии RMSD), но уже через 3–4 эпохи кривые сближаются, а затем сеть без предобучения идёт ниже (рисунок 1). Для достижения порога $\text{val loss} \leq 0,55$ нужно 6,6 эпохи без предобучения и 7,6 эпохи с ним; минимум val loss равен $0,395 \pm 0,004$ и $0,406 \pm 0,006$ соответственно. Колебательность (2,27 против 2,42 %; $p = 0,098$) и шум (1,95 против 2,07 %; $p = 0,39$) у предобученной сети чуть ниже, но различие в пределах разброса между запусками.

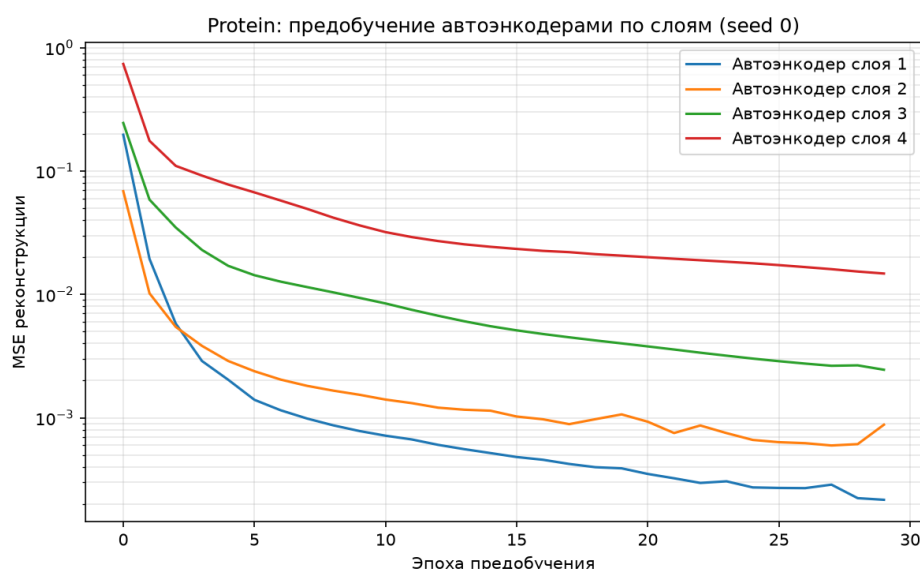


Рисунок 2 – Ошибка реконструкции автоэнкодеров при послойном предобучении (Protein, seed 0)

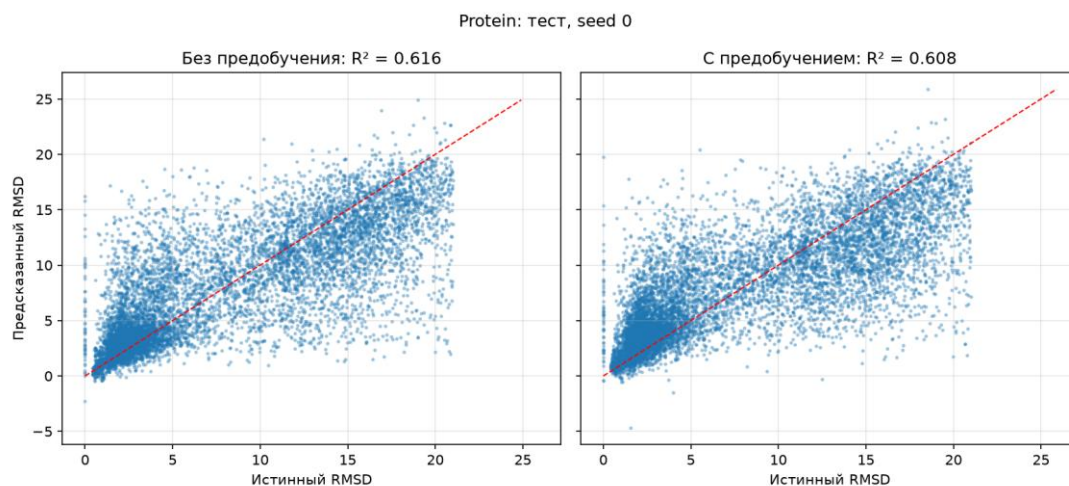


Рисунок 3 – Предсказанный и истинный RMSD на тесте (Protein, seed 0)

Влияние функции активации

Сеть и автоэнкодеры строились с одной и той же функцией активации скрытых слоёв (3 запуска):

Активация	Режим	RMSE, Å	R ²	MAPE, %	Колебательность, %	Шум, %
ReLU	Без предобучения	$3.836 \pm$	$0.604 \pm$	$52.681 \pm$	2.503 ± 0.199	$2.142 \pm$

		0.057	0.012	3.583		0.339
ReLU	С предобучением	3.867 ± 0.044	0.598 ± 0.009	56.466 ± 1.949	2.347 ± 0.375	2.126 ± 0.660
Tanh	Без предобучения	3.920 ± 0.045	0.586 ± 0.009	55.669 ± 1.112	1.354 ± 0.203	1.085 ± 0.200
Tanh	С предобучением	3.893 ± 0.031	0.592 ± 0.007	56.076 ± 1.557	1.167 ± 0.256	0.942 ± 0.204
Logistic (sigmoid)	Без предобучения	4.662 ± 0.057	0.415 ± 0.015	87.035 ± 4.021	0.754 ± 0.099	0.728 ± 0.134
Logistic (sigmoid)	С предобучением	4.435 ± 0.020	0.471 ± 0.004	77.757 ± 1.689	1.031 ± 0.033	0.901 ± 0.128

При ReLU и Tanh предобучение результат не меняет. С логистической функцией (как в лекции) сеть обучается заметно хуже (RMSE 4,66 против 3,84 у ReLU), и именно здесь предобучение помогает: RMSE 4,43 против 4,66, R^2 0,47 против 0,42. Это согласуется с теорией: логистическая функция усиливает затухание градиента, и осмысленная начальная точка частично компенсирует эту проблему, но не устраняет разрыв с ReLU.

Малые обучающие выборки

Эксперимент повторён на случайных подвыборках обучающей части (валидационная и тестовая выборки прежние, 5 запусков, ReLU):

Объектов в train	Режим	RMSE, Å	R^2	MAPE, %
250	Без предобучения	5.235 ± 0.143	0.266 ± 0.037	99.076 ± 5.864
	С предобучением	5.230 ± 0.140	0.267 ± 0.039	99.213 ± 5.737
1000	Без предобучения	4.858 ± 0.057	0.368 ± 0.015	81.051 ± 2.268
	С предобучением	4.908 ± 0.085	0.355 ± 0.021	88.770 ± 5.198
5000	Без предобучения	4.379 ± 0.049	0.487 ± 0.012	69.996 ± 1.269
	С предобучением	4.436 ± 0.055	0.473 ± 0.012	71.508 ± 2.977
27438	Без предобучения	3.871 ± 0.063	0.599 ± 0.011	53.897 ± 3.729
	С предобучением	3.908 ± 0.065	0.591 ± 0.013	56.434 ± 1.643

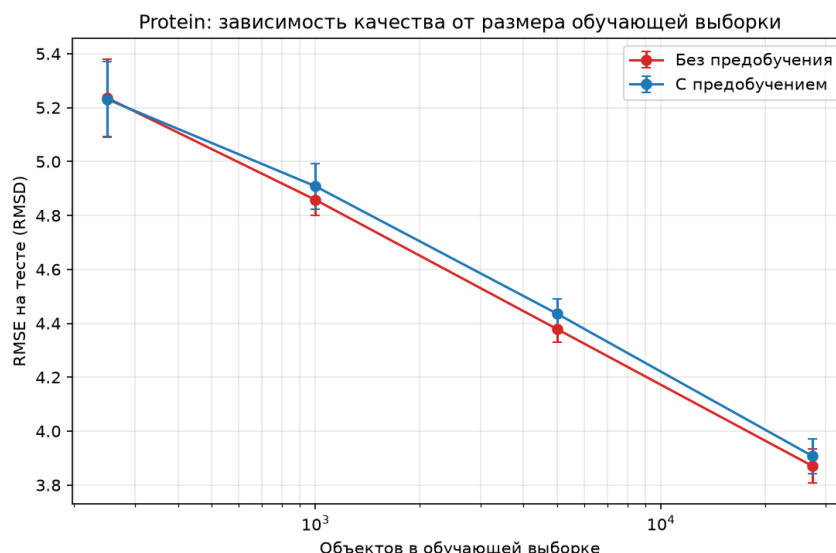


Рисунок 4 – Зависимость RMSE на тесте от размера обучающей выборки (Protein)

Ни при 250, ни при 1 000, ни при 5 000 объектах предобучение преимущества не дало (при 250 объектах RMSE 5,230 и 5,235 – совпадают). Вероятная причина: автоэнкодеры восстанавливают 9 входных признаков, а зависимость RMSD от них сложная и слабо связана с структурой самих признаков, поэтому информация, добытая без учителя, для задачи почти бесполезна.

2. WDBC (датасет из ЛР №2, классификация)

Качество на тестовой выборке, среднее $\pm$ СКО по 10 запускам (положительный класс – М):

Режим	Efficiency, %	F1	Precision	Recall
Без предобучения	96.842 $\pm$ 1.812	0.957 $\pm$ 0.025	0.963 $\pm$ 0.036	0.953 $\pm$ 0.035
С предобучением	96.404 $\pm$ 1.129	0.951 $\pm$ 0.015	0.963 $\pm$ 0.033	0.941 $\pm$ 0.030

Парный t-тест: Efficiency $p = 0,177$; F1 $p = 0,172$; Precision $p = 0,924$; Recall $p = 0,177$ – значимых различий нет. Матрицы ошибок, суммарные по 10 тестовым выборкам (строки – истина, столбцы – прогноз):

Режим	В $\rightarrow$ В	В $\rightarrow$ М	М $\rightarrow$ В	М $\rightarrow$ М
Без предобучения	700	16	20	404
С предобучением	700	16	25	399

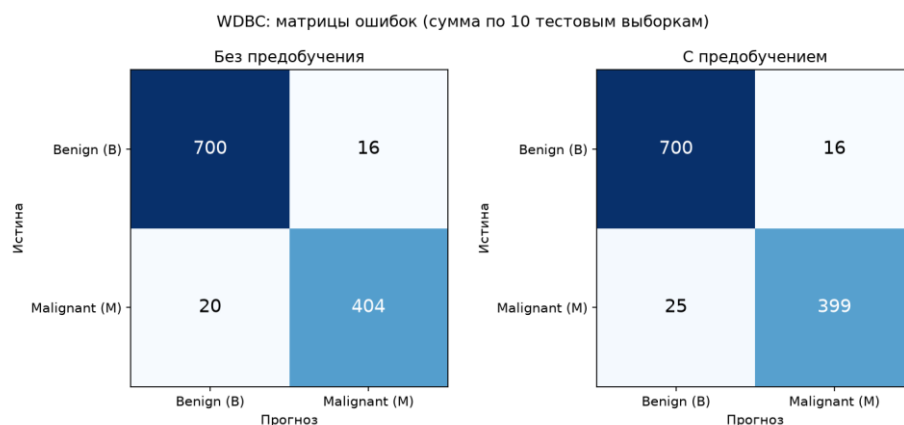


Рисунок 5 – Матрицы ошибок (WDBC, сумма по 10 тестовым выборкам)

Число ложных срабатываний одинаково (16), а пропусков злокачественных опухолей чуть больше у предобученной сети (25 против 20 из 424), что не значимо.

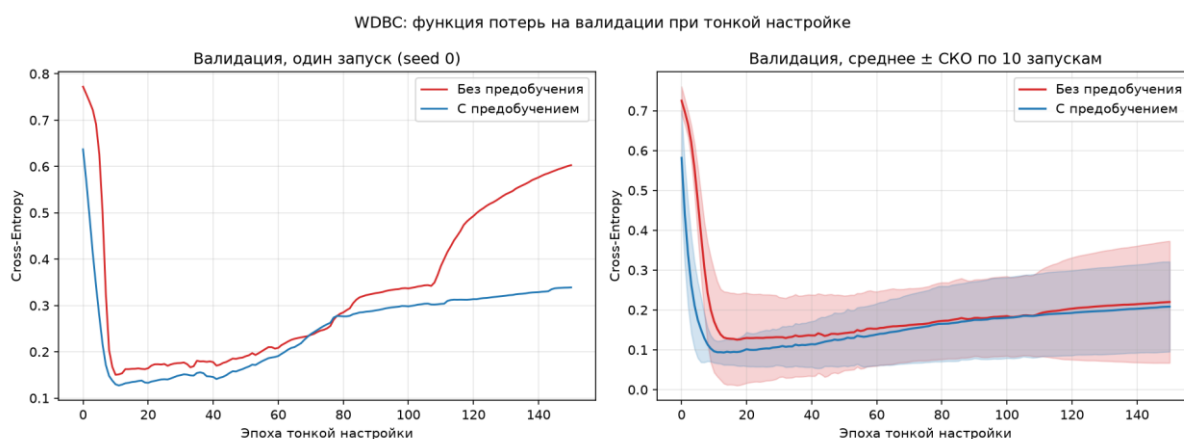


Рисунок 6 – Ошибка (Cross-Entropy) на валидации при тонкой настройке (WDBC)

Динамика обучения:

Режим	Val loss, эпоха 0	Лучшая эпоха	Колебательность, %	Шум val-кривой, %	Эпох с ростом val, %
Без предобучения	0.725 ± 0.037	43.500 ± 53.757	2.505 ± 4.049	3.394 ± 6.612	74.429 ± 22.345
С предобучением	0.582 ± 0.124	22.900 ± 21.553	1.424 ± 1.060	1.649 ± 1.549	79.214 ± 14.100

Предобученная сеть стартует с меньшей ошибкой (0,582 против 0,725) и быстрее обучается: порог $\text{Cross-Entropy} \leq 0,2$ достигается за 4,6 эпохи против 8,7 (без предобучения один из 10 запусков порога не достиг); минимальная val loss $0,083 \pm 0,038$ против $0,098 \pm 0,080$ – меньше и по значению, и по разбросу между запусками. После 10–20-й эпохи на малой выборке (341 объект) начинается переобучение – val loss растёт у обеих сетей (рисунок 6), поэтому важен выбор лучшей эпохи по валидации. Колебательность (1,42 против 2,51

%) и шум (1,65 против 3,39 %) у предобученной сети ниже в среднем, но из-за большого разброса между запусками различия незначимы ($p = 0,33$ и $0,35$).

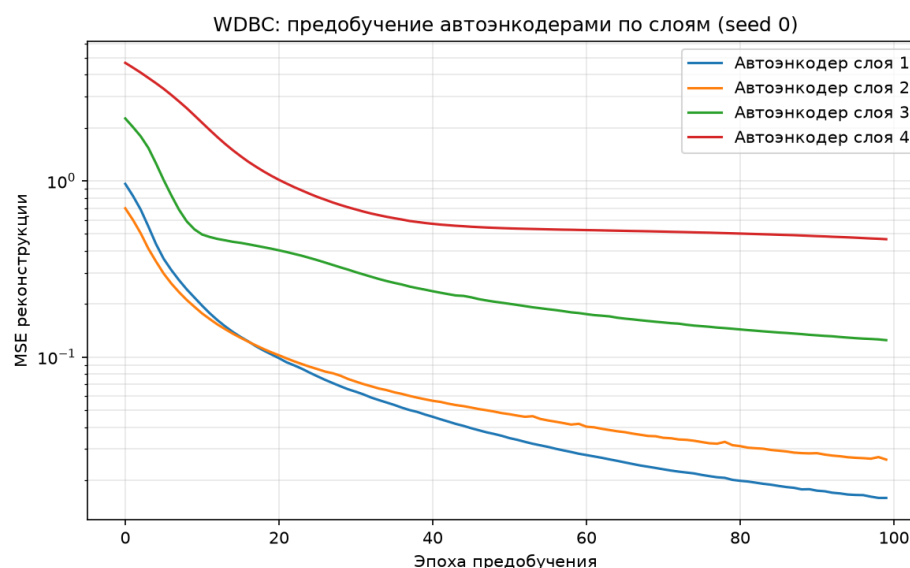


Рисунок 7 – Ошибка реконструкции автоэнкодеров при предобучении (WDBC, seed 0)

Влияние функции активации

5 запусков:

Активация	Режим	Efficiency, %	F1	Колебательность, %	Шум, %
ReLU	Без предобучения	96.316 ± 1.902	0.950 ± 0.026	1.491 ± 1.435	1.684 ± 2.502
ReLU	С предобучением	96.140 ± 1.468	0.947 ± 0.020	1.392 ± 1.225	1.651 ± 1.957
Tanh	Без предобучения	97.018 ± 1.710	0.959 ± 0.024	2.013 ± 1.438	2.680 ± 2.033
Tanh	С предобучением	96.667 ± 2.587	0.953 ± 0.038	2.472 ± 1.771	3.570 ± 2.430
Logistic (sigmoid)	Без предобучения	97.544 ± 1.144	0.966 ± 0.016	1.991 ± 0.242	0.438 ± 0.162
Logistic (sigmoid)	С предобучением	96.667 ± 1.687	0.953 ± 0.026	2.318 ± 0.837	0.869 ± 0.694

На небольшой табличной выборке все три функции активации дают сопоставимое качество (≈ 96 – $97,5$ %), предобучение его не повышает; с логистической функцией кривая валидации заметно глаже (шум $0,44$ – $0,87$ % против $\approx 1,7$ % у ReLU), с Tanh – более шумная ($2,7$ – $3,6$ %).

Малые обучающие выборки

Подвыборки обучающей части (валидационная и тестовая прежние, 10 запусков, ReLU):

Объектов в train	Режим	Efficiency, %	F1
30	Без предобучения	88.333 ± 18.331	0.882 ± 0.127

	С предобучением	94.211 ± 2.930	0.922 ± 0.038
60	Без предобучения	89.035 ± 18.448	0.893 ± 0.127
	С предобучением	95.175 ± 1.861	0.936 ± 0.021
120	Без предобучения	96.053 ± 1.324	0.947 ± 0.017
	С предобучением	96.053 ± 0.745	0.947 ± 0.009
341	Без предобучения	96.842 ± 1.812	0.957 ± 0.025
	С предобучением	96.404 ± 1.129	0.951 ± 0.015

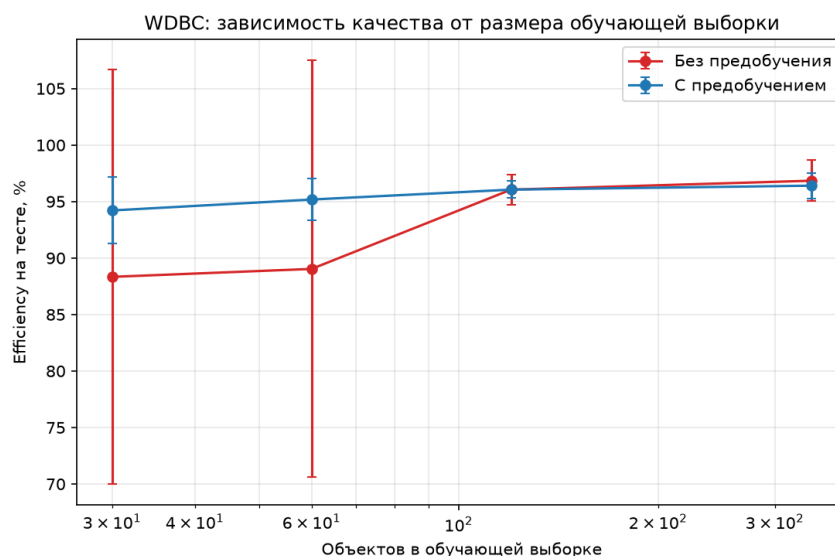


Рисунок 8 – Зависимость Efficiency на тесте от размера обучающей выборки (WDBC)

При 30 и 60 объектах средняя точность с предобучением заметно выше (94,2 и 95,2 % против 88,3 и 89,0 %), а разброс между запусками много меньше (СКО 2,9 и 1,9 п.п. против 18,3 и 18,4 п.п.). Однако пошаговая проверка запусков показала, что причина – один запуск без предобучения (seed 6), в котором сеть «умерла» и дала 36,8 % (предсказывает всем объектам один класс); тот же запуск с предобучением дал 95,6 % (при 30 объектах) и 96,5 % (при 60). Без этого запуска средние совпадают ($\approx 94,1$ % при 30 объектах как с предобучением, так и без него). Следовательно, на малых выборках предобучение в нашем эксперименте повышает не точность в типичном случае, а надёжность обучения: оно защищает от неудачной инициализации. При 120 и более объектах режимы равноценны (96,05 % против 96,05 %).

3. Сравнение результатов обучения с предобучением и без него

Показатель	Protein (регрессия)	WDBC (классификация)
Итоговое качество на тесте	без предобучения чуть лучше: RMSE 3,871 против 3,908 Å (≈ 1 %), R^2 0,599 против 0,591	различий нет: Efficiency 96,84 против 96,40 %, F1 0,957 против 0,951 ($p \approx 0,17$)
Стартовая val loss (эпоха 0)	ниже у предобученной: 0,769 против 1,013	ниже у предобученной: 0,582 против 0,725
Скорость сходимости	не ускоряется (7,6 против 6,6 эпохи до порога)	ускоряется: 4,6 против 8,7 эпохи до порога

Сглаживание val-кривой	незначимое (колебательность 2,27 против 2,42 %, $p = 0,10$)	тенденция есть (1,42 против 2,51 %), но незначима ($p = 0,33$)
Малые выборки	преимущества нет (250 объектов: 5,230 против 5,235 Å)	выше устойчивость при 30–60 объектах (избегает «мёртвого» запуска); при ≥ 120 объектов различий нет
Логистическая активация	предобучение помогает: RMSE 4,43 против 4,66 Å	различий нет

Общий вывод по пункту 3: автоэнкодерное предобучение не улучшает итоговое качество сети с ReLU, когда обучающих данных достаточно (Protein, 27 438 объектов; WDBC, 341 объект). Его реальные эффекты – более выгодная стартовая точка (меньшая начальная ошибка), более быстрая сходимость на WDBC, помощь при сложной для обучения логистической активации и защита от неудачной инициализации на очень малых выборках. Это согласуется с лекцией: метод обратного распространения с ReLU хорошо работает при большой обучающей выборке, а предобучение предназначено для малых выборок. Для выборки Protein автоэнкодеры, восстанавливающие 9 признаков, не извлекают информации, полезной для предсказания RMSD, поэтому выигрыша нет.

Вопросы по варианту

1. Учитывая размерность выборки физико-химических свойств белка и целевую переменную RMSD, какова была архитектура глубокой сети (более 3 слоев) и какие функции активации применялись в автоэнкодерах?
2. Произошло ли сглаживание процесса обучения (уменьшение колебаний функции потерь на валидации) при использовании инициализации весов через автоэнкодер?

1) Архитектура и активации. Выборка содержит 9 входных признаков (F1–F9) и одну непрерывную целевую переменную RMSD (регрессия, 45 730 объектов), поэтому входной слой имеет 9 нейронов, выходной – один линейный нейрон без функции активации (диапазон RMSD не ограничен сигмоидой). Использована полносвязная глубокая сеть 9-128-64-32-16-1: четыре скрытых слоя (128, 64, 32 и 16 нейронов, сужающиеся по мере подъёма абстракции) и выходной слой, т.е. пять слоёв с весами, что больше трёх. Функция потерь – MSE, оптимизатор Adam.

Для предобучения построены четыре автоэнкодера: 9-128-9, 128-64-128, 64-32-64 и 32-16-32. В кодирующей части каждого автоэнкодера (он же соответствующий слой итоговой сети) применена функция активации ReLU, реконструирующий слой – линейный (без активации), потому что

восстанавливаются стандартизованные вещественные значения, которые могут быть отрицательными (а у второго и последующих автоэнкодеров входом служат неотрицательные выходы ReLU предыдущего слоя). Критерий обучения автоэнкодеров – MSE реконструкции (30 эпох, Adam, $lr = 0,001$). Дополнительно проверены Tanh и логистическая функция (сеть и автоэнкодеры с одной и той же активацией): ReLU дала лучшее качество (RMSE 3,84 Å, против 3,92 у Tanh и 4,66 у Logistic без предобучения), поэтому она принята в качестве основной.

2) Сглаживание обучения. Однозначного сглаживания не произошло. Колебательность val-кривой (средний модуль изменения val loss между эпохами) составила $2,27 \pm 0,30$ % у предобученной сети против $2,42 \pm 0,19$ % без предобучения (-6 %, $p = 0,098$), шум относительно скользящего среднего – $1,95 \pm 0,53$ % против $2,07 \pm 0,26$ % (-6 %, $p = 0,39$). Доля эпох, на которых val loss растёт, практически одинакова (43,8 % и 45,3 %). Для Tanh колебания у предобученной сети меньше (1,17 против 1,35 %), а для логистической активации – больше (1,03 против 0,75 %); оба результата получены на трёх запусках и в пределах разброса. Единственное устойчивое отличие – более низкая стартовая ошибка на валидации (0,769 против 1,013), но оно исчезает за 3–4 эпохи. Итак, на выборке Protein инициализация весов через автоэнкодер заметного сглаживания процесса обучения не дала; на WDBC тенденция к более гладкой кривой с меньшим разбросом между запусками есть (шум 1,65 против 3,39 %), но она статистически незначима ($p \approx 0,35$).

Вывод

Вывод: научилась осуществлять предобучение нейронных сетей с помощью автоэнкодерного подхода. Реализовано жадное послойное предобучение глубокой полносвязной сети (четыре скрытых слоя + выходной) автоэнкодерами на PyTorch с последующим обучением выходного слоя и тонкой настройкой всей сети, и выполнено сравнение с обучением без предобучения на выборке Protein Tertiary Structure (вариант 6, регрессия, RMSD) и на выборке WDBC из ЛР №2 (классификация). Для обеих выборок эксперименты повторены на нескольких разбиениях, различия проверены парным t-тестом. На полных выборках предобучение итоговое качество не улучшило (Protein: RMSE 3,908 против 3,871 Å, $R^2 \approx 0,6$; WDBC: Efficiency 96,4 против 96,8 %), что соответствует лекции: при достаточном объёме данных и функции активации ReLU метод обратного распространения справляется и без предобучения. Предобучение дало более низкую стартовую ошибку, ускорило сходимость на WDBC, улучшило обучение сети с логистической активацией (RMSE 4,43 против 4,66 Å) и повысило устойчивость обучения на очень малых

выборках (30–60 объектов WDBC). Существенного сглаживания кривой потерь на валидации на выборке Protein не обнаружено.